

Sorting learning notes

Round 4

1. Quicksort（快速排序）

核心：选 pivot，把数组 partition 成两边： $< \text{pivot}$ 在左， $\geq \text{pivot}$ 在右；然后递归排左右两边。

例：[2 4 5 1 3 6 5 2]，pivot=5 $\Rightarrow$ [2 4 1 3 2 | 5 | 6 5]。

本质：divide & conquer。

复杂度：平均 $O(n \log n)$ ，最坏 $O(n^2)$ （比如 pivot 总选到最小/最大导致极度不平衡）。

优点：通常很快、cache friendly、in-place；

缺点：最坏 $O(n^2)$ ，重复值多时性能会变差（用 3-way 可优化）。

2. Mergesort（归并排序）

核心：先分再合并。把数组不断对半拆到单元素，再把两个“已排序子数组”用双指针 merge 成一个有序数组。

复杂度：永远 $O(n \log n)$ ；

缺点：需要额外空间 $O(n)$ 。

算法	平均	最坏	空间
Quicksort	$O(n \log n)$	$O(n^2)$	$O(1)$
Mergesort	$O(n \log n)$	$O(n \log n)$	$O(n)$

3. Counting Sort（计数排序）

核心：不比较元素，直接统计出现次数。

$V = [2, 4, 5, 1, 3, 6, 5, 2, 1, 5, 4, 3, 4, 3, 6, 2]$

设取值范围 $0..r$ ，建计数数组 $C[0..r]$ ：

1) 计数： $C[x] = x$ 出现次数；

$C = [0, 2, 3, 3, 3, 3, 2]$

```

For i = 0 to r:
    C[i] = 0
For i = 0 to n-1:
    C[V[i]] = C[V[i]] + 1

```

2) 前缀和： $C[x] = \leq x$ 的元素个数 (cumulative sum)，用来定位每个值在输出数组的位置区间；

cumulative count 给的是“最后一个位置”

例如： $C[5]=10$ 意思 最后一个 5 应该放在 index 9

$C = [0, 2, 5, 8, 11, 14, 16]$

```

For i = 1 to r:
    C[i] = C[i] + C[i-1]

```

3) 放入新数组：通常从后往前扫以保证 stability (稳定性)。

$U = [1, 1, 2, 2, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5, 5, 5, 6, 6]$

$C = [0, 0, 2, 5, 8, 11, 14]$

```

For i = n-1 to 0:
    U[C[V[i]]-1] = V[i]
    C[V[i]] = C[V[i]] - 1

```

复杂度： $O(n + r)$ 。

限制：只适合整数且范围 r 不能太大 (否则空间/时间爆炸)。

4. String Sorting (字符串排序)

关键点：字符串比较不是 $O(1)$ ，最坏要比较到第一个不同字符，每次比较代价 $\sim O(w)$ ($w = \text{string length}$)。

最坏： $O(n \log n \cdot w)$ 。

平均： $O(n \log n \cdot \log n)$

5. MSD Radix Sort (字符串的“按位分组”排序)

MSD = Most Significant Digit：

从最左字符开始分组 (类似“字符串版 counting sort”)，每组再按下一位递归分。

“一层”的时间大概是： $O(n + r + n)$

复杂度常写成 $O(w n + w r)$: w = 最大字符串长度, n = 字符串个数, r = alphabet size (ASCII 256 ; 小写字母 26)

这是上界, 很多实现不会在每个子问题都付出完整的 r (会有优化/复用), 而且并不是每层都对所有 n 都扫描到。

6. 3-way Quicksort (处理大量重复值)

普通 quicksort 在重复值很多时会继续递归, 浪费。

3-way 直接分三组 : $<p$ 、 $=p$ 、 $>p$; 中间 $=p$ 的那段不再递归, 所以 **duplicates** 多时更快。

7. 3-way Quicksort for Strings

把 3-way quicksort 和 radix 的“按字符位”结合 : 比较当前字符 d 。

左/右两组继续用同一个 d ; 中间组 (当前字符相等) 用 $d+1$ 继续比下一位。

```
Parameters:
  V: an array of n strings
  s and e: the start and end indices of the current subarray to sort
  d: the current character comparison index
1 StringQuick3(V, s, e, d):
2   If s < e:                                     does the subarray contain more than 1 item?
3     pi = random index within s...e             pi is the pivot index; selected randomly
4     p = V[pi]                                   p is the pivot string
5     swap V[s] and V[pi]                         move pivot initially to the start
6     eqs = s                                     eqs tells where the middle group starts
7     eqe = e                                     eqe will in the end tell where the middle group ends
8     i = s+1                                     i starts to scan items from the right side of the pivot
9     While i <= eqe:
10      If V[i][d] < p[d]:                         only characters at index d are compared
11        swap V[eqs] and V[i]                     move string V[i] to the left group
12        eqs += 1                                  adding V[i] to left pushes middle group start to right
13        i += 1
14      Else If V[i][d] > p[d]:
15        swap V[eqe] and V[i]                     move string V[i] to the right group
16        eqe -= 1                                  adding V[i] to right pushes middle group end to left
17      Else:
18        i += 1                                     the characters matched
19        StringQuick3(V, s, eqs-1, d)              add (= keep) V[i] in the middle group
20        If length of p is > d:                    sort the left group recursively
21          StringQuick3(V, eqs, eqe, d+1)          did index d exist in the pivot?
22          StringQuick3(V, eqe+1, e, d)            sort the middle group using d+1
23          sort the right group recursively
```

最小可能的最大递归深度 smallest possible maximum recursion depth

```
V = [ CTGCTT, CATATA, ATTAAT, TTA CTC, AATGCG,
      ATTGTC, CTCCGT, GTGCAA, AAGTGG, TACTAT,
      AGATAT, GTTGAG, ATACAC, CTCCGC, CAGGCA,
      AACAAA, TTACTA, CATGCA, AGTACT, AAGACC]
```

只需要找：最难分开的字符串

也就是：最长公共前缀（Longest Common Prefix）

TTACTA 和 TTACTC 公共前缀是5

最深 recursive chain：

1. Level 1: 全数组
2. Level 2: T...
3. Level 3: TT...
4. Level 4: TTA...
5. Level 5: TTAC...
6. Level 6: TTACT...
7. Level 7: TTACTA / TTACTC
8. Level 8: 完成单字符串